

Computational Complexity of Sorting Algorithms and Space Efficiency

Introduction to Algorithm Complexity Analysis

Sorting algorithms are foundational procedures in computer science that arrange items. The computational efficiency of an algorithm is analyzed using asymptotic Big-O notation. Both worst-case time complexity and auxiliary space overhead are decisive metrics.

Algorithm Efficiency and Resource Consumption Table:

Algorithm	Time Complexity	Space Complexity
Merge Sort	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$ average	$O(\log n)$
Bubble Sort	$O(n^2)$	$O(1)$
Heap Sort	$O(n \log n)$	$O(1)$

Auxiliary Space and In-Place Sorting Characteristics

Notice that Merge Sort requires $O(n)$ auxiliary space because of temporary subarrays. In contrast, Bubble Sort and Heap Sort operate strictly in-place with $O(1)$ auxiliary space. Quick Sort requires $O(\log n)$ auxiliary stack space for its recursive partitioning stack. Stability: Merge Sort and Bubble Sort are stable, while Quick Sort and Heap Sort are not.